

Reviewer Pack — Minimal Rank of Algebraic Curves Bounds Monotone Circuit Low...

Entry 3b9771098b75 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 23:11:30 UTC

Minimal Rank of Algebraic Curves Bounds Monotone Circuit Lower Bound for k-CLIQUE

Entry ID: 3b9771098b75 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-28 23:11:30 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Algebraic Curves) **Field B** (complexity object): Complexity Theory: Monotone Circuit Complexity

Statement:

{‘st_1’: ‘Let F be a monotone DNF formula representing the k -CLIQUE problem, and let $C(F)$ denote its algebraic curve complexity. For all constant $c > 0$, there exists a polynomial $p_c(n)$ such that for every n -vertex k -CLIQUE instance, $C(F) \leq p_c(n)$.’, ‘st_2’: ‘The polynomial $p_c(n)$ is at least $\Omega(n^k)$, where k is the size of the largest clique in the instance.’, ‘st_3’: ‘For a fixed k , the ratio $E[C(F)/n^k]$ converges to 1 as n grows large.’}

Rationale (proposer’s reasoning):

{‘ra_1’: ‘Algebraic curves have been used to study computational problems, but their connection with monotone circuit complexity is underexplored.’, ‘ra_2’: ‘This conjecture suggests that the algebraic structure of k -CLIQUE instances could provide insights into the complexity of monotone circuits.’, ‘ra_3’: ‘If true, it would imply a new approach to proving monotone circuit lower bounds for k -CLIQUE.’}

Taxonomy category: MONOTONE_CLIQUE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): f118a1638280f8fa

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

If the polynomial fit’s mean curve complexity ratio $E[C(F)/n^k]$ is within $\pm 10\%$ of 1 for all $n \leq 40$, and no seed yields a $C(F)/n^k$ value exceeding 1.1.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	UNCERTAIN	0.90	UNCERTAIN	HITS
ALGEBRIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "algebraic curve complexity" AND "monotone circuit lower bound" - "polynomial $p_c(n)$ " AND "k-CLIQUE problem" AND "algebraic geometry" - "ratio $E[C(F)/n^k]$ " AND "convergence to 1" AND "monotone DNF formula"

Top relevant hits considered: - [s2:2012.03883] Monotone Circuit Lower Bounds from Robust Sunflowers - [s2:2109.06941] Monotone Complexity of Spanning Tree Polynomial Re-visited

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_clique_instance(n, k):
    if n < k:
        return None, 0, []

    vertices = list(range(n))
    edges = [(i, j) for i in range(n) for j in range(i + 1, n) if (i, j) not in vertices[:k]]
    random.shuffle(edges)
    edges = edges[:n * k // 2]

    return vertices, k, edges

def matrix_multiplication(A, B):
    m, p = len(A), len(B[0])
    result = [[sum(A[i][j] * B[j][k] for j in range(p)) for k in range(p)] for i in range(m)]
    return result

def gaussian_elimination(matrix):
    n = len(matrix)
    augmented_matrix = [row[:] + [1 if i == j else 0 for j in range(n)] for i, row in enumerate(matrix)]

    for i in range(n):
        max_row = max(range(i, n), key=lambda r: abs(augmented_matrix[r][i]))
        augmented_matrix[i], augmented_matrix[max_row] = augmented_matrix[max_row], augmented_matrix[i]

        pivot = augmented_matrix[i][i]
        if pivot == 0:
            return None
```

```

    for j in range(n):
        augmented_matrix[i][j] /= pivot

    for r in range(n):
        if r != i:
            factor = augmented_matrix[r][i]
            for j in range(n):
                augmented_matrix[r][j] -= factor * augmented_matrix[i][j]

    return [row[n:] for row in augmented_matrix]

def algebraic_curve_complexity(vertices, edges):
    n = len(vertices)
    adjacency_matrix = [[0] * n for _ in range(n)]
    for u, v in edges:
        adjacency_matrix[u][v] = 1
        adjacency_matrix[v][u] = 1

    rank = len(gaussian_elimination(adjacency_matrix))
    return rank

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [5, 10, 15, 20, 30, 40]
    curve_complexities = []

    for n in n_values:
        k = min(n // 2, 5) # Ensure k is at least 1 and at most n/2
        vertices, _, edges = generate_clique_instance(n, k)

        if vertices is None:
            return {
                "metric_name": "C(F)/n^k",
                "metric_value": None,
                "instances_tested": 0,
                "conjecture_holds": False,
                "counterexample": "k > n"
            }

        complexity = algebraic_curve_complexity(vertices, edges)
        curve_complexities.append(complexity / (n ** k))

    mean_value = sum(curve_complexities) / len(curve_complexities)
    std_value = math.sqrt(sum((x - mean_value) ** 2 for x in curve_complexities) / len(curve_complexities))

    return {
        "metric_name": "C(F)/n^k",
        "metric_value": mean_value,
        "instances_tested": len(n_values),
        "conjecture_holds": abs(mean_value - 1) <= 0.1 and max(curve_complexities) <= 1.1,
        "counterexample": ""
    }

if __name__ == "__main__":

```

```

import sys
seeds = [int(x) for x in sys.argv[1:]] if sys.argv[1:] else [2**i + 3 for i in range(5, 8)]

results = []
for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\"}, \"metric_value\": {result['metric_value']}}")
    results.append(result)

mean_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(results)
std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results if r["metric_value"] is not None) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] or r["metric_value"] is None for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] and r["metric_value"] is not None for r in results):
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"C(F)/n^k > 1.1\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE no data points")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0f06ca4d.py", line 106, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0f06ca4d.py", line 86, in run_trial
    complexity = algebraic_curve_complexity(vertices, edges)
                  ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0f06ca4d.py", line 64, in algebraic_curve_complexity
    rank = len(gaussian_elimination(adjacency_matrix))
           ~~~~~
TypeError: object of type 'NoneType' has no len()

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from verifying the conjecture's conditions. | next: Investigate and fix the error in the test code to allow for a proper verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12950
2	preregistration	ollama_remote	glm4:latest	0	0	5861
3	novelty	ollama_remote	glm4:latest	0	0	5118
4	novelty	ollama_remote	glm4:latest	0	0	6389
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16822
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11441
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12161
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13632
9	judge	ollama_remote	glm4:latest	0	0	9153

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 93526 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/3b9771098b75.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/3b9771098b75.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/3b9771098b75.tar.gz` (if generated)